

# PointSplat: Unified Image-Agnostic Gaussian Pruning and 3D Transformer Refinement

Anh Thuan Tran, Jana Kořecká  
Department of Computer Science  
George Mason University  
{atran92, kosecka}@gmu.edu

## Abstract

*3D Gaussian Splatting (3DGS) has recently advanced the field of novel view synthesis, enabling real-time rendering of high-fidelity images using explicit 3D Gaussian representations. However, traditional methods often require millions of Gaussians to capture complex scenes, leading to significant memory and storage demands. Recent approaches have addressed this issue through pruning and per-scene fine-tuning of Gaussian parameters, thereby reducing the number of Gaussians while maintaining visual quality. These strategies typically rely on 2D images to compute important scores followed by scene-specific optimization. In this work, we introduce PointSplat, 3D unified prune-and-refine framework that bridges previously disjoint directions of image-driven pruning and transformer refinement. Our method includes two key components: (1) an intrinsic pruning strategy that ranks Gaussians based solely on their 3D attributes, removing reliance on 2D images; and (2) a balanced dual-branch encoder that explicitly separates and re-weights geometric and appearance features to avoid feature dominance. Extensive experiments on ScanNet++ and Replica across varying sparsity levels demonstrate that PointSplat consistently achieves competitive rendering quality and improved efficiency without additional per-scene optimization.*

## 1. Introduction

Novel view synthesis (NVS) aims to render photorealistic images of 3D scenes from unseen viewpoints. This capability has broad applications across various fields, including virtual reality (VR), augmented reality (AR) [4], robotics [30], and autonomous driving [10]. Neural Radiance Fields (NeRF) [18] reconstructs 3D representations from multi-view images by mapping 3D coordinates and viewing directions to view-dependent color and volume density. The pixel intensities are computed through volume rendering

techniques [1]. While NeRF and its variants have achieved high-quality results, they suffer from slow inference speed, limiting their deployment in real-time or interactive applications.

Recent advances in 3D Gaussian Splatting (3DGS) [13] have emerged as a promising alternative, enabling real-time rendering while maintaining competitive visual fidelity. 3DGS represented scenes through explicit 3D Gaussians, which were rendered using splatting techniques [14], with each Gaussian parameterized by position, scale, rotation, opacity, and coefficients of spherical harmonics modeling the viewpoint dependent appearance of the scene. While 3D Gaussians are typically initialized using point clouds from 3D reconstructions, due to the densification process and overparameterization [7], the number of Gaussians can grow to millions per scene, resulting in significant memory demands and slowdown in rendering performance. To address this issue, several studies have proposed methods to reduce the number of Gaussians required for high-quality rendering through pruning based on importance scores. LightGaussians [7] quantified Gaussian’s contribution to each pixel ray across all training views, while Mini-Splatting [9] used densification and simplification algorithm to reorganize the spatial positions of Gaussians rather than directly pruning them. PUP-3DGS [12] improved the scoring method using sensitivity-based pruning for higher compression ratios, while Speedy-Splat [11] introduced hard and soft pruning algorithms to further accelerate rendering and reduce memory requirements. In prior approaches [7, 11, 12], pruning scores were typically computed using 2D images, along with associated Gaussian attributes and corresponding camera rays. Although these strategies enhance rendering quality with fewer Gaussians, they rely on access to 2D images and necessitate per-scene optimization. Consequently, deploying them to novel scenes requires both pretrained 3DGS models and corresponding 2D images for score computation and fine-tuning. This dependency limits their applicability in downstream scenarios where obtaining test-time images or per-scene tuning is impractical for secu-

rity or privacy reasons.

In this work, we propose *PointSplat*, a transformer-based framework for image-agnostic pruning and refinement of sparse 3D Gaussians. Our pruning score is derived solely from 3D attributes, balancing spatial coverage and opacity to select a compact yet informative subset of Gaussians. The retained Gaussians are then refined by Balanced Dual-Branch Encoder, which separates geometric attributes (position, scale, orientation) from high-dimensional appearance features (spherical harmonics) and reweights them with positional encoding for structured refinement. Unlike prior approaches that addressed pruning and refinement in isolation, PointSplat unifies these directions into a single 3D pipeline. Image-driven pruning [7, 11, 12] relied on 2D images and per-scene optimization, while transformer-only refinement [6] suffered from feature imbalance due to appearance dominance. To our best knowledge, PointSplat is the first framework to combine intrinsic 3D pruning with balanced refinement, enabling structured refinement of compact Gaussians without requiring images at deployment. Our main contributions can be described as follows:

- We introduce a novel opacity–volume pruning score that selects informative Gaussians purely from 3D attributes, enabling image-agnostic pruning that provides compact representations while preserving spatial coverage and visual quality.
- We propose PointSplat, a 3D pruning-and-refinement network with Balanced Dual-Branch Encoder that separates geometric information from high-dimensional appearance features, reinforcing them through 3D position encoding.
- We conduct extensive experiments on ScanNet++ and Replica, demonstrating that PointSplat consistently outperforms over 3D transformer-based 3DGS baseline across multiple sparsity levels, validating its robustness and deployment-time generalization.

## 2. Related Work

Our work relates to improving the efficiency and practicality of pruning 3D Gaussian Splatting (3DGS) by addressing Gaussian redundancy and the limitations of existing pruning strategies, that rely on 2D images and scene-specific optimization.

**3D Representations.** In Neural Radiance Fields (NeRF) [18], 3D scenes are represented using camera rays, and multi-layer perceptrons (MLPs) are used to map 3D coordinates and viewing directions to color and density values. This representation provides high-quality novel view synthesis through volume rendering along each ray. However, since NeRF renders each pixel by independently querying an MLP, it is typically scene specific and suffers from slow inference speed, making it impractical for real-time applications. To overcome speed limitation, several stud-

ies have proposed alternative spatial representations to accelerate NeRF-based rendering. For instance, TensorRF [3] modeled radiance fields as 4D tensors for efficient computation, while Instant-NGP [19] utilized hash-based encoding to achieve faster convergence and real-time performance. To build 3D representations of unseen scenes, pixelNeRF [28] conditioned radiance fields on sparse sets of input images and trained a single model for multiple scenes, while IBRNet [23] used a ray-transformer network that jointly encoded 3D locations and 2D viewing directions to synthesize novel views from few images. Although these methods represented significant progress, achieving consistent real-time rendering—particularly for large-scale or complex scenes remains a challenge for NeRF-based approaches. Recent advances in point-based rendering via 3D Gaussian Splatting (3DGS) [13] have opened new directions for efficient novel view synthesis with competitive visual quality. 3DGS represented 3D scenes using large number (millions) of 3D Gaussians, each with position, orientation, scale, color, and opacity, followed by rendering novel views by directly projecting (splatting) these Gaussians into image space and blending their contributions [14], enabling efficient real-time rendering. To improve novel view synthesis from sparse set of views, pixelSplat [2] sampled Gaussian means from dense probability distribution over 3D obtained by predefined encoder, while MVSplat [5] localized Gaussian centers using cost volume approach. MVSGaussians [17] further leveraged multi-view stereo to encode point-wise features, which were then decoded into pixel-aligned Gaussian parameters, providing more effective initialization and faster fine-tuning. Building on these advances, our method focuses on refining pretrained 3D Gaussians by leveraging their explicit 3D positions and geometric attributes. As a result, this approach enables pruning and refinement without requiring additional 2D images for score computation at inference stage.

**Gaussian Ranking and Pruning.** While 3D Gaussian Splatting (3DGS) achieves remarkable rendering performance, it also suffers from redundancy in the number of Gaussians [7], resulting in high memory demands. Several methods have been proposed to prune the model and reduce non-essential parameters while preserving rendering quality and improving efficiency. Compact-3DGS [16] introduced a codebook-based approach that groups similar Gaussians based on geometry; LightGaussians [7] improved significance score calculation to include global factors such as hit count from each pixel, along with vector quantization and distillation of spherical harmonics parameters. PUP-3DGS [12] evaluated the sensitivity of each Gaussian to the rendered image and prunes accordingly. Mini-Splatting [9] enhanced the densification and simplification stages by assigning an importance score to each Gaussian during optimization. In parallel, Speedy-Splat [11] focuses on accel-

erating rendering through specialized rasterization kernels, while PointSplat operates at the representation level. Thus, PointSplat and Speedy-Splat are complementary rather than directly comparable. Most existing pruning strategies compute importance scores from 2D images paired with 3D models and require scene specific fine-tuning. To address these limitations, PointSplat introduces a novel pruning score that relies solely on intrinsic 3D Gaussian attributes. By prioritizing spatially informative splats and discarding redundant ones with high spatial overlap, our approach reduces redundancy and improves rendering efficiency. The retained Gaussians are further refined by 3D transformer-based network to improve structural fidelity and have better coverage. Crucially, at inference time, PointSplat operates directly on pretrained 3DGS models without requiring additional 2D images or per-scene fine-tuning.

**Point Cloud Processing Refinements.** Recent advancements in 3D Gaussian Splatting (3DGS) have opened several opportunities for using point cloud understanding and processing architectures for further refinement of the Gaussian parameters. Instead of hand-crafted regularization techniques [26], several approaches adapted attention-based point transformer [15, 22, 24, 25, 29] to process the 3DGS as a point cloud set with Gaussian attributes serving as features. SplatFormer [6] applied PointTransformerV3 [25] to refine 3D Gaussians, improving representation quality for out-of-distribution tasks. Large Spatial Model (LSM) [8] used pretrained 2D transformer and 3D point-based transformer models to predict 3D Gaussians from two unposed input images.

Previous efforts faced challenges when directly applying point-based transformer models to 3D Gaussians. Most point cloud transformer models were originally designed for traditional point clouds, where each point typically contains only a small number of attributes (XYZ coordinates and RGB color). LSM [8] addressed this issue by computing appearance through point-wise aggregation with color from stereo point maps, omitting high-dimensional spherical harmonics coefficients, a critical factor in 3DGS for modeling view-dependent appearance. To address this, SplatFormer [6] concatenated all Gaussian parameters—including opacity, scale, rotation, spherical harmonics, and position—into a single feature vector and feeds it into PointTransformerV3 [25]. However, this approach suffered from feature imbalance: spherical harmonics coefficients can be up to 48 dimensions (for  $L = 3$ ), significantly outweighing other features like opacity (1D), scale, rotation, or position (all 3D), leading to appearance-dominated representations that may lower effective learning.

In contrast, PointSplat introduces Balanced Dual-Branch Encoder. This explicitly separates geometric and appearance features, reweights them with positional encoding, and prevents appearance dominance. This module directly ad-

dresses previous issues with transformer-only refinement, enabling balanced learning across attributes.

### 3. Method

In this work, we present PointSplat, a prune-and-refine framework that unifies Gaussian pruning and transformer-based refinement into a single 3D pipeline with image-agnostic pruning. Our approach addresses two key challenges: (1) selecting informative Gaussians that capture scene structure without relying on 2D images, and (2) refining the retained Gaussians while balancing geometric and appearance features. Specifically, we introduce a novel pruning score that balances spatial coverage and opacity, selecting Gaussians that are both compact and representative. The retained Gaussians are then processed by Balanced Dual-Branch Encoder and 3D point-based transformer, which refines their parameters by integrating local geometric context with appearance features, obtaining compact yet expressive scene representations.

#### 3.1. Preliminaries

3D Gaussian Splatting (3DGS) [13] represents a scene using 3D Gaussians  $G = \{G_i\}_{i=1}^N$  where  $N$  is total number of Gaussians. Each Gaussian  $G_i$  is defined by its mean position  $\mu_i \in \mathbb{R}^3$  and covariance matrix  $\Sigma_i \in \mathbb{R}^{3 \times 3}$ . Given 3D point  $\mathbf{x}$ , each Gaussian can be defined as:

$$G_i(\mathbf{x}) \propto \exp\left(-\frac{1}{2}(\mathbf{x} - \mu_i)^T \Sigma_i^{-1}(\mathbf{x} - \mu_i)\right) \quad (1)$$

with covariance matrix  $\Sigma_i = R_i S_i S_i^T R_i^T$ , rotation matrix  $R_i$  parametrized by quaternion  $q_i \in \mathbb{R}^4$ , and diagonal scaling matrix  $S_i$  derived from scaling vector  $\mathbf{s}_i = [s_i^x, s_i^y, s_i^z]$ . Each Gaussian has associated appearance attributes, including opacity value  $\alpha_i \in \mathbb{R}$  and color represented by coefficients of spherical harmonics (SH) basis functions. Specifically, the color  $c_i$  is computed from SH coefficients  $\gamma(i) \in \mathbb{R}^{(L+1)^2 \times 3}$ :

$$c_i = \sum_{l=0}^L \sum_{m=-l}^l \gamma_l^m(i) Y_l^m(\mathbf{v}_i) \quad (2)$$

where  $L$  denotes SH degree,  $\mathbf{v}_i$  is the viewing direction, and  $Y_l^m$  is the spherical harmonic basis function of degree  $l$  and order  $m$ . While spherical harmonics of higher degree  $L$  can effectively capture view-dependent appearance, they also significantly increase the dimensionality of appearance attributes in 3DGS representations. At  $L = 3$ , each Gaussian requires 16 coefficients per color channel, totaling 48 coefficients. This increase in the model’s parameters can lead to an imbalance between the geometric (position) and appearance features of each Gaussian in the subsequent learning stage. During training [14] parameters

associated with the Gaussians are optimized end-to-end per scene using gradient descent to minimize a photometric loss over the input images. To improve rendering quality, densification stage splits, clones and prunes corresponding Gaussians. While this process increases reconstruction fidelity, it often expands the number of Gaussians from thousands to millions, leading to increased storage requirements.

### 3.2. Gaussian Pruning Score

Our first objective is to select the most informative subset of Gaussians that preserves both the global structure and appearance of the scene. To ensure reliable representation, the chosen Gaussians should maintain sufficient spatial coverage so that the geometry of the scene is well captured, while the opacity determines Gaussian’s contribution to visual appearance. To balance geometric coverage and appearance contribution, we propose a pruning score that combines Gaussian volume (derived from scale vector  $s_i$ ) and opacity  $\alpha_i$ . The weighted sum of these two terms is controlled by a trade-off parameter  $\lambda_\alpha$ :

$$\text{score}_i = \lambda_\alpha \cdot \nu(\alpha_i) + (1 - \lambda_\alpha) \cdot \nu\left(\frac{4}{3}\pi s_i^x s_i^y s_i^z\right) \quad (3)$$

where  $\nu(\alpha_i)$  and  $\nu(\frac{4}{3}\pi s_i^x s_i^y s_i^z)$  denote the z-score normalized opacity and volume of  $i^{th}$  Gaussian and is computed for all  $N$  Gaussians. The weighting factor  $\lambda_\alpha \in [0, 1]$  controls the balance between appearance and spatial contribution. The top  $K$  Gaussians are selected based on this score, prioritizing Gaussians that provide broad spatial coverage and contribute significantly to visual quality. Since our refinement stage is built upon a transformer architecture operating on sparse 3D points, spatial coverage is a critical factor. A well-distributed selection of Gaussians ensures that the network receives enough information from diverse spatial regions, enabling it to effectively learn scene structure and appearance from the multiple images of complex indoor environments.

### 3.3. Balanced Dual-Branch Encoder

Given sparse set  $K$  of Gaussians selected in the previous stage, we next describe how to refine their parameters. Instead of using standard local gradient-based optimization (like Adam), we use 3D point-based transformer [25] that globally coordinates the updates of Gaussians across the entire scene. While parameters in 3D Gaussian Splatting share some similarities with traditional 3D point clouds, they differ in several important ways. In various learning tasks on 3D point clouds, each point typically is characterized by 3D coordinates (XYZ) and RGB color—features that are low-dimensional and of same dimensionality. Prior works [24, 25, 29] often concatenate these features directly as input to transformer-based architectures. In

contrast, Gaussians in 3DGS have with diverse and high-dimensional attributes, including 3D position  $\mu_i$ , scale  $s_i$ , rotation  $q_i$  (as quaternion), opacity  $\alpha_i$ , and spherical harmonics (SH) coefficients  $\gamma_i$  for view-dependent color representation. Among these, SH coefficients are particularly high-dimensional and simply concatenating all Gaussian parameters into a single feature vector, as done in SplatFormer [6], the dominance of the SH coefficients can lead to feature imbalance. This can overshadow geometric factors, leading to biased feature learning and suboptimal model performance.

To address this, we introduce a dual-branch encoder for parameters associated with Gaussians that balances appearance and geometry related features before feeding them into the point transformer (see Figure 1b). The encoder consists of two parallel branches: *geometric branch*, which processes the 3D coordinates  $\mu_i$ , scale  $s_i$ , and rotation  $q_i$ ; and *appearance branch*, which processes the opacity  $\alpha_i$  together with reduced color features obtained by applying an MLP  $\varphi$  to the spherical harmonics coefficients,  $\varphi(\gamma_i)$ .

$$f_i^{(p)} = \text{concat}(\mu_i, q_i, s_i) \quad f_i^{(a)} = \text{concat}(\alpha_i, \varphi(\gamma_i)) \quad (4)$$

$$\delta_i = \psi(\mu_i) \quad (5)$$

$$f_i = \sigma(\phi_a(f_i^{(a)}) + \delta_i) \odot (\phi_p(f_i^{(p)}) + \delta_i) \quad (6)$$

Each branch is passed through separate MLPs  $\phi_p, \phi_a$  to extract corresponding geometric and appearance features (see Figure 1b). Additionally, we introduce position encoding module to reinforce 3D spatial locations in input features. Unlike geometric branch, which includes intrinsic properties such as scale and rotation, the position encoding module  $\psi$  uses solely explicit 3D coordinates  $\mu_i$ . This module ensures that we focus only the physical 3D locations of Gaussians when refining both geometric and appearance-related features, avoiding confusion with intrinsic Gaussian attributes, such as scale or rotation. The appearance features, combined with position encoding, are then transformed using softmax function  $\sigma$  yielding an attention weight for each Gaussian. These weights are applied to geometric features through point-wise multiplication, after adding position encoding to them as well.

The balanced dual-branch encoder serves two key purposes: (1) it mitigates the dominance of high-dimensional appearance features, preventing them from overwhelming geometric information; and (2) it enriches geometric features with appearance information and spatial location, resulting in more balanced and effective representations.

**3D Transformer Network.** Given the encoded features, we leverage 3D point transformer network [25] to optimize the appearance and geometric attributes of selected Gaussians. There are two core motivations behind adopting a point transformer architecture in this work. First, as illustrated in

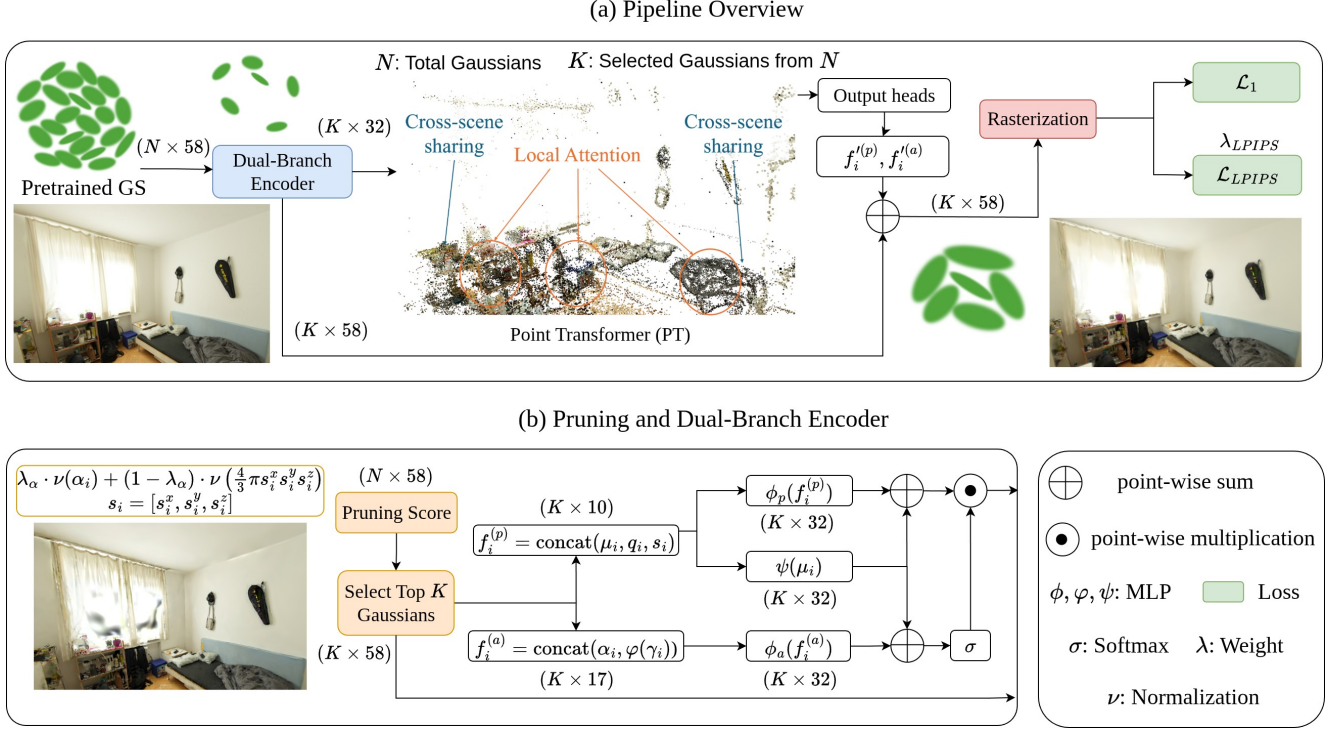


Figure 1. **Overall architecture.** (a) The pipeline begins with pretrained Gaussian Splatting (GS) model. Gaussians are first scored by the Dual-Branch Encoder (b), which selects an informative subset. Selected Gaussians are then encoded through separate geometry and appearance branches with positional encoding. These features are refined by Point Transformer and integrated with the pruned Gaussians. Finally, the sparse refined Gaussians are rasterized to render images and compute the training loss.

Figure 1a, point transformers employ local attention mechanisms, allowing each point to refine its features based on spatial neighbors. This locality constraint preserves fine-grained structural details while preventing influence from distant, unrelated Gaussians. Second, by operating on individual Gaussians with explicit 3D coordinates and enriched geometric features, the network can capture common structural characteristics across scenes, leading to more effective refinement of sparse Gaussians.

We treat the selected Gaussians as static 3D points, similar to point clouds, where each point is defined by its position and learned feature vector. These features are first processed through point transformer network (PT), and then passed through separate MLP heads to get corresponding position and appearance Gaussian parameters. Finally, residual connections are applied to update the original Gaussian attributes before rasterization.

$$f_i^{(p)}, f_i^{(a)} = \text{PT}(f_i) \quad (7)$$

$$f_i^{(p)} = f_i^{(p)} + f_i^{(p)}, \quad f_i^{(a)} = f_i^{(a)} + f_i^{(a)} \quad (8)$$

Furthermore, by learning features across multiple scenes, PointSplat network can refine sparse Gaussians that share

similar properties (e.g., floors, walls), leading to better spatial coverage, as shown in Figure 2. This refinement improves scene quality and can be applied directly to pretrained 3DGS models without additional fine-tuning.

## 4. Experimental Results

**Datasets.** We evaluate the proposed network on ScanNet++ [27] using the novel view synthesis task, with 5 held-out test scenes. For Replica [20], we follow the setup in [31] and evaluate on 3 held-out test scenes. In both datasets, following prior works [6, 13], 10% of the images in each scene are reserved for testing, while the remaining 90% are used to obtain initial pretrained 3DGS models. We choose ScanNet++ and Replica due to its availability of dense multi-view data, which is essential for generating high-quality pretrained 3DGS models. While our PointSplat network requires only two images per scene for refinement—similar to prior works [2, 5]—it is built upon pretrained 3DGS models that are trained using all available views. Other datasets used in these works provide only sparse supervision and do not offer sufficient multi-view coverage for robust 3DGS pretraining. Therefore, evaluating on such datasets is not directly compatible with our pipeline.

**Implementation details.** We implement our architecture

Table 1. Experimental results on ScanNet++ and Replica under varying Gaussian sparsity levels  $G$  (%). Methods are grouped into *Scene-Specific / Image-Driven* (require 2D images and per-scene optimization) and *Image-Agnostic* (operate without images at deployment). \* denote pruned Gaussians evaluated directly, without refinement through fine-tuning (scene-specific) or 3D networks (image-agnostic).

| $G$ (%)                                      | Method                    | Scannet++       |                 |                    | Replica         |                 |                    |
|----------------------------------------------|---------------------------|-----------------|-----------------|--------------------|-----------------|-----------------|--------------------|
|                                              |                           | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ | PSNR $\uparrow$ | SSIM $\uparrow$ | LPIPS $\downarrow$ |
| 100%                                         | 3DGS (Dense)              | 29.73           | 0.925           | 0.165              | 39.40           | 0.974           | 0.122              |
| <b>Scene-Specific / Image-Driven Methods</b> |                           |                 |                 |                    |                 |                 |                    |
| 50%                                          | LightGaussian*            | 28.19           | 0.913           | 0.188              | 32.33           | 0.957           | 0.143              |
|                                              | LightGaussian             | 30.23           | 0.929           | 0.171              | <b>39.78</b>    | <b>0.974</b>    | 0.125              |
|                                              | PUP-3DGS*                 | 28.29           | 0.913           | 0.185              | 32.44           | 0.955           | 0.158              |
|                                              | PUP-3DGS                  | <b>31.35</b>    | <b>0.937</b>    | <b>0.159</b>       | 39.67           | 0.973           | <b>0.121</b>       |
| 30%                                          | LightGaussian*            | 26.13           | 0.891           | 0.215              | 26.86           | 0.921           | 0.182              |
|                                              | LightGaussian             | 29.80           | 0.923           | 0.183              | 38.63           | <b>0.970</b>    | 0.135              |
|                                              | PUP-3DGS*                 | 24.80           | 0.882           | 0.224              | 23.20           | 0.896           | 0.200              |
|                                              | PUP-3DGS                  | <b>30.85</b>    | <b>0.931</b>    | <b>0.172</b>       | <b>38.64</b>    | 0.968           | <b>0.130</b>       |
| 10%                                          | LightGaussian*            | 21.87           | 0.836           | 0.278              | 20.29           | 0.837           | 0.290              |
|                                              | LightGaussian             | 28.12           | 0.902           | 0.224              | 33.37           | 0.937           | 0.195              |
|                                              | PUP-3DGS*                 | 19.82           | 0.803           | 0.305              | 17.34           | 0.784           | 0.322              |
|                                              | PUP-3DGS                  | <b>29.64</b>    | <b>0.915</b>    | <b>0.205</b>       | <b>36.04</b>    | <b>0.951</b>    | <b>0.169</b>       |
| <b>Image-Agnostic Methods</b>                |                           |                 |                 |                    |                 |                 |                    |
| 50%                                          | SplatFormer               | 21.27           | 0.857           | 0.348              | 30.79           | 0.927           | 0.233              |
|                                              | PointSplat*               | 28.81           | 0.923           | 0.266              | 33.97           | 0.956           | 0.213              |
|                                              | PointSplat* + SplatFormer | 29.03           | 0.924           | 0.264              | 34.63           | 0.952           | 0.192              |
|                                              | PointSplat                | <b>29.52</b>    | <b>0.928</b>    | <b>0.258</b>       | <b>35.98</b>    | <b>0.958</b>    | <b>0.191</b>       |
| 30%                                          | SplatFormer               | 11.66           | 0.691           | 0.509              | 29.14           | 0.903           | 0.263              |
|                                              | PointSplat*               | 27.15           | 0.909           | <b>0.287</b>       | 30.01           | 0.934           | 0.245              |
|                                              | PointSplat* + SplatFormer | 26.66           | 0.899           | 0.300              | 33.04           | 0.939           | <b>0.207</b>       |
|                                              | PointSplat                | <b>27.70</b>    | <b>0.911</b>    | <b>0.287</b>       | <b>34.46</b>    | <b>0.947</b>    | 0.208              |
| 10%                                          | SplatFormer               | 10.62           | 0.691           | 0.569              | 8.37            | 0.388           | 0.677              |
|                                              | PointSplat*               | 21.88           | 0.838           | 0.377              | 17.60           | 0.802           | 0.397              |
|                                              | PointSplat* + SplatFormer | 22.59           | 0.847           | 0.375              | 27.99           | 0.888           | <b>0.271</b>       |
|                                              | PointSplat                | <b>23.46</b>    | <b>0.855</b>    | <b>0.374</b>       | <b>29.81</b>    | <b>0.902</b>    | 0.276              |

using PyTorch on NVIDIA A100 80GB. We begin by obtaining pretrained 3DGS models (Splatfacto) from NerfStudio [21], which provide 3D Gaussian positions along with corresponding features, including opacity, scale, rotation, and spherical harmonics coefficients.

The proposed network is trained for 10,000 iterations using the Adam optimizer. The loss function is combined from  $\mathcal{L}_1$  loss with a weighted  $\mathcal{L}_{\text{LPIPS}}$  Learned Perceptual Image Patch Similarity (LPIPS) loss ( $\lambda_{\text{LPIPS}} = 0.1$ ), following [6], and is defined as  $\mathcal{L} = \mathcal{L}_1 + \lambda_{\text{LPIPS}} \mathcal{L}_{\text{LPIPS}}$ . The learning rate is initialized to  $1e-5$  and dropped by 10 at iteration 6,000. In the downsampling stages of the Point Transformer, we configure a 5-stage hierarchical architecture, with the number of transformer blocks in each stage set to (2, 2, 2, 4, 2), respectively. Each MLP layer contains linear layers fol-

lowed by batch normalization and ReLU activation. We fix  $\lambda_\alpha = 0.3$  for both datasets; results remain stable across a range of values, demonstrating robustness.

**Evaluation metrics.** We follow standard practice of [6, 13] and report PSNR (Peak Signal-to-Noise Ratio), SSIM (Structured Similarity Index Metric), and LPIPS (Learned Perceptual Image Patch Similarity) metrics to evaluate rendering quality.

#### 4.1. Quantitative Results

The pruning score alone already provides large improvements, significantly boosting 3DGS and SplatFormer under heavy pruning, as shown in Table 1. For example, SplatFormer applied to Gaussians pruned with our score (denoted *PointSplat\** + *SplatFormer*) increases PSNR from

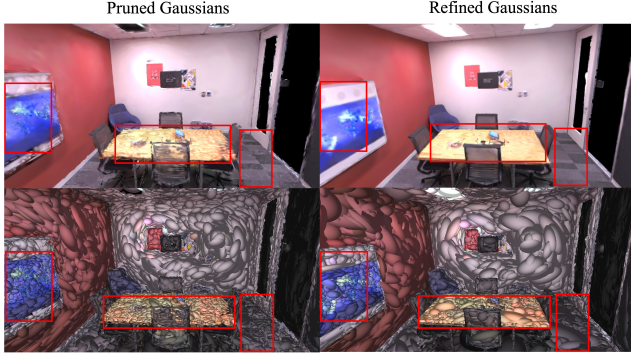


Figure 2. **Comparison of pruned Gaussians (left) and refined Gaussians (right).** Pruning alone often leads to artifacts such as blurred textures, broken edges, and incomplete geometry. In contrast, PointSplat refinement adjusts the retained Gaussians to capture structural details, producing compact ellipsoids that align with scene geometry.

21.27 to 29.03 on ScanNet++ at 50% sparsity, and from 8.37 to 27.99 on Replica at 10%. These results confirm that pruning alone selects more informative Gaussians. PointSplat further enhances reconstruction quality; Balanced Dual-Branch Encoder effectively addresses feature imbalance beyond pruning-only strategies. Importantly, without fine-tuning, our pruning score surpasses most scene-specific pruning strategies in the same setting. While scene-specific methods can exceed PointSplat after per-scene refinement, our approach delivers strong performance immediately, without additional images or optimization, highlighting its efficiency and image-agnostic nature. In contrast to LightGaussian and PUP-3DGS, which require recomputing scores for each scene, PointSplat applies the same learned refinement model to test scenes without re-training or per-scene adaptation.

While PointSplat shows slightly higher LPIPS than *PointSplat\** + *SplatFormer* in some cases (e.g., 0.276 vs. 0.271 on Replica at 10%), these differences are marginal compared to its consistently superior PSNR (27.99  $\rightarrow$  29.81) and SSIM (0.888  $\rightarrow$  0.902). This trade-off indicates that PointSplat emphasizes geometric fidelity—recovering sharper structures and edges—while maintaining competitive perceptual quality on both datasets. Figure 2 and Table 1 also highlight the broader significance of unifying pruning and refinement: pruning alone produces artifacts, while transformer-only refinement suffers from appearance dominance. PointSplat’s unified framework achieves a synergistic effect with compact, high-opacity Gaussians refined into structurally accurate representations that neither direction achieves in isolation.

## 4.2. Ablation study

Ablation studies are conducted on 10% sparsity level.

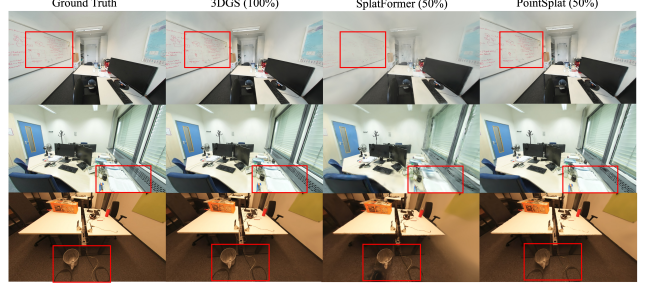


Figure 3. **Qualitative comparison.** Ground truth images, pre-trained dense 3DGS, and reconstructions from SplatFormer and PointSplat with 50% of Gaussians. PointSplat delivers sharper details and more precise structures, preserves fine textures and complex shapes across scenes, achieving higher perceptual and structural fidelity without per-scene fine-tuning.

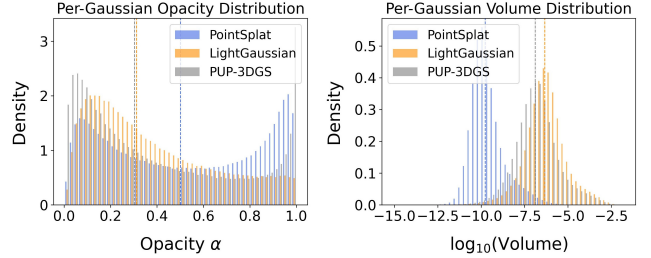


Figure 4. **Distribution of Gaussian opacity (left) and log-volume (right) at 50% pruning.** Compared to LightGaussian (orange) and PUP-3DGS (gray), PointSplat (blue) concentrates on Gaussians with higher opacity and smaller volumes. This contrasts with other studies, which leave large but weak splats.

**Efficiency.** Table 2 presents efficiency comparisons between our method and prior works. At equal storage and Gaussian count, PointSplat achieves consistently higher FPS than prior pruning methods at all sparsity levels. These results demonstrate that pruning and refinement can provide structurally compact Gaussians with higher opacity, accelerating rendering. Figure 4 further analyzes the per-Gaussian statistics, showing that PointSplat achieves smaller Gaussians (median log-volume  $\approx -10$ ) with higher opacity (median  $\approx 0.5$ ), confirming the compact yet expressive nature of our representation.

**Choosing Gaussians.** Table 3 and Figure 5 show ablation study on pruning score weight  $\lambda_\alpha$ , which controls the trade-off between appearance (opacity) and spatial coverage (Gaussian volume). We emphasize that  $\lambda_\alpha = 0.3$  is fixed globally across all datasets rather than tuned per dataset. This setting provides a balanced trade-off, thereby providing the highest PSNR and SSIM. Extremely low values (e.g.,  $\lambda_\alpha = 0.1$ ) overemphasize volume and select spatially large but visually weak Gaussians, while very high values (e.g.,  $\lambda_\alpha = 0.9$ ) overemphasize opacity and reduce spatial coverage, both of which degrade quality. Moreover,

Table 2. Efficiency results:  $\#G$  = number of Gaussians.

| $G$ (%) | Method        | $\#G$ | FPS           | Size(MB) |
|---------|---------------|-------|---------------|----------|
| 100%    | 3DGS (Dense)  | 588K  | 207.41        | 145.92   |
| 50%     | LightGaussian | 294K  | 213.30        | 72.96    |
|         | PUP-3DGS      | 294K  | 278.26        | 72.96    |
|         | PointSplat    | 294K  | <b>390.51</b> | 72.96    |
| 30%     | LightGaussian | 176K  | 253.42        | 43.78    |
|         | PUP-3DGS      | 176K  | 352.71        | 43.78    |
|         | PointSplat    | 176K  | <b>440.56</b> | 43.78    |
| 10%     | LightGaussian | 59K   | 356.04        | 14.58    |
|         | PUP-3DGS      | 59K   | 579.98        | 14.58    |
|         | PointSplat    | 59K   | <b>621.53</b> | 14.58    |

Table 3. Ablation on  $\lambda_\alpha$ : PointSplat\* denotes pruned Gaussians before refinement, and PointSplat after transformer refinement.

| $\lambda_\alpha$ | Method      | ScanNet++    |              |              | Replica      |              |              |
|------------------|-------------|--------------|--------------|--------------|--------------|--------------|--------------|
|                  |             | PSNR         | SSIM         | LPIPS        | PSNR         | SSIM         | LPIPS        |
| 0.1              | PointSplat* | 20.52        | 0.824        | 0.411        | 22.42        | 0.846        | 0.364        |
|                  | PointSplat  | 21.72        | 0.837        | 0.406        | 28.23        | 0.881        | 0.281        |
| 0.3              | PointSplat* | 21.87        | 0.836        | 0.377        | 17.60        | 0.802        | 0.397        |
|                  | PointSplat  | <b>23.46</b> | <b>0.855</b> | <b>0.374</b> | <b>29.81</b> | <b>0.902</b> | 0.276        |
| 0.5              | PointSplat* | 20.57        | 0.813        | 0.402        | 15.45        | 0.752        | 0.433        |
|                  | PointSplat  | 22.72        | 0.846        | 0.376        | 27.71        | 0.888        | <b>0.274</b> |
| 0.7              | PointSplat* | 17.55        | 0.751        | 0.453        | 13.90        | 0.700        | 0.462        |
|                  | PointSplat  | 20.10        | 0.816        | 0.410        | 27.39        | 0.886        | 0.281        |
| 0.9              | PointSplat* | 13.74        | 0.621        | 0.516        | 12.48        | 0.643        | 0.487        |
|                  | PointSplat  | 10.66        | 0.686        | 0.559        | 26.58        | 0.882        | 0.290        |

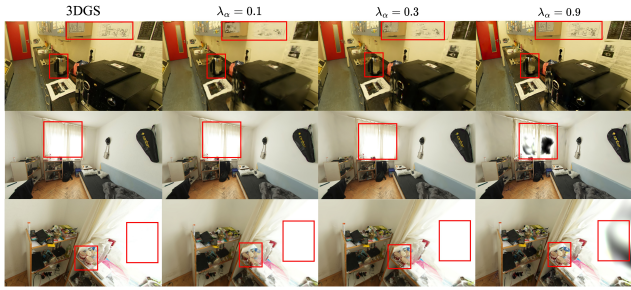


Figure 5. **Effect of pruning weight  $\lambda_\alpha$ .** We visualize the pre-trained 3DGS model before pruning and subsets selected with different  $\lambda_\alpha$ . A balanced setting ( $\lambda_\alpha = 0.3$ ) maintains both spatial coverage and visual details. Lower values overemphasize volume and lose fine details, while higher values overemphasize opacity and produce uneven distributions. This demonstrates the need to balance geometry and appearance for robust sparse rendering.

PointSplat outperforms the baseline when selected Gaussians are informative, demonstrating its strong refinement capability. These results confirm that PointSplat remains robust across a range of  $\lambda_\alpha$  values and does not rely on sensitive hyperparameter tuning.

**Balanced Dual-Branch Encoder.** Table 4 presents an ablation study of the proposed dual-branch encoder. Model I serves as the baseline, using pruned 3DGS without refinement. Model II uses geometry features  $f_i^{(p)}$ , such as scale and rotation. Model III adds position encoding  $\delta_i$  to the geometry branch. Model IV uses appearance features  $f_i^{(a)}$  as weights applied to geometry features, but excludes position encoding. Finally, Model V uses both geometry and appearance features, each augmented with position encoding. The results demonstrate that using geometry features alone (Model II) improves performance over the baseline (Model I). Adding position encoding to the geometry branch (Model III) yielding further improvement, especially in SSIM and LPIPS. However, combining geometry and ap-

Table 4. Ablation on the dual-branch encoder.

| ID  | $f_i^{(p)}$ | $f_i^{(a)}$ | $\delta_i$ | PSNR         | SSIM         | LPIPS        |
|-----|-------------|-------------|------------|--------------|--------------|--------------|
| I   |             |             |            | 21.87        | 0.836        | 0.377        |
| II  | ✓           |             |            | 22.61        | 0.848        | 0.384        |
| III | ✓           |             | ✓          | 22.76        | 0.847        | 0.385        |
| IV  | ✓           | ✓           |            | 22.65        | 0.846        | 0.388        |
| V   | ✓           | ✓           | ✓          | <b>23.46</b> | <b>0.855</b> | <b>0.374</b> |

pearance without any positional encoding (Model IV) leads to weaker results, showing that spatial context is essential when fusing features. Model V achieves the best overall performance across all metrics, demonstrating the effectiveness of using geometry and appearance, both enhanced with position encoding, to improve network understanding of spatial in sparse 3D Gaussian refinement.

**Limitation.** Our approach is particularly effective on datasets that are structurally similar (e.g. indoors) and extension to outdoor or large-scale environments is left for future work. What distinguishes PointSplat is that its deployment-time refinement requires no additional images or scene-specific tuning, making it more efficient and practical in scenarios where test images are unavailable.

## 5. Conclusion

We propose PointSplat, a 3D framework that unifies pruning and transformer-based refinement for sparse Gaussians. Our pruning score selects compact and informative Gaussians, while spatially-aware dual-branch encoder balances geometry and appearance during refinement. Experiments on ScanNet++ (real-world) and Replica (synthetic) demonstrate that PointSplat achieves competitive performance across different sparsity levels without per-scene tuning. Overall, its compact representation is well suited for indoor applications such as AR/VR, and robotics, where reducing model size and accelerating deployment are critical.

## References

- [1] Robert A Brebin, Loren Carpenter, and Pat Hanrahan. Volume rendering. In *Seminal graphics: pioneering efforts that shaped the field*, pages 363–372. 1998. [1](#)
- [2] David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 19457–19467, 2024. [2](#), [5](#)
- [3] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. Tensorf: Tensorial radiance fields. In *European conference on computer vision*, pages 333–350. Springer, 2022. [2](#)
- [4] Guikun Chen and Wenguan Wang. A survey on 3d gaussian splatting. *arXiv preprint arXiv:2401.03890*, 2024. [1](#)
- [5] Yuedong Chen, Haoqi Xu, Chuanxia Zheng, Bohan Zhuang, Marc Pollefeys, Andreas Geiger, Tat-Jen Cham, and Jianfei Cai. Mvsplat: Efficient 3d gaussian splatting from sparse multi-view images. In *European Conference on Computer Vision*, pages 370–386. Springer, 2024. [2](#), [5](#)
- [6] Yutong Chen, Marko Mihajlovic, Xiyi Chen, Yiming Wang, Sergey Prokudin, and Siyu Tang. Splatformer: Point transformer for robust 3d gaussian splatting. In *ICLR*, 2025. [2](#), [3](#), [4](#), [5](#), [6](#)
- [7] Zhiwen Fan, Kevin Wang, Kairun Wen, Zehao Zhu, Dejia Xu, Zhangyang Wang, et al. Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps. *Advances in neural information processing systems*, 37: 140138–140158, 2024. [1](#), [2](#)
- [8] Zhiwen Fan, Jian Zhang, Wenyan Cong, Peihao Wang, Renjie Li, Kairun Wen, Shijie Zhou, Achuta Kadambi, Zhangyang Wang, Danfei Xu, et al. Large spatial model: End-to-end unposed images to semantic 3d. *Advances in neural information processing systems*, 37:40212–40229, 2024. [3](#)
- [9] Guangchi Fang and Bing Wang. Mini-splatting: Representing scenes with a constrained number of gaussians. In *European Conference on Computer Vision*, pages 165–181. Springer, 2024. [1](#), [2](#)
- [10] Ben Fei, Jingyi Xu, Rui Zhang, Qingyuan Zhou, Weidong Yang, and Ying He. 3d gaussian splatting as new era: A survey. *IEEE Transactions on Visualization and Computer Graphics*, 2024. [1](#)
- [11] Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. *arXiv preprint arXiv:2412.00578*, 2024. [1](#), [2](#)
- [12] Alex Hanson, Allen Tu, Vasu Singla, Mayuka Jayawardhana, Matthias Zwicker, and Tom Goldstein. Pup 3d-gs: Principled uncertainty pruning for 3d gaussian splatting. *arXiv preprint arXiv:2406.10219*, 2024. [1](#), [2](#)
- [13] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023. [1](#), [2](#), [3](#), [5](#), [6](#)
- [14] Georgios Kopanas, Julien Philip, Thomas Leimkühler, and George Drettakis. Point-based neural rendering with per-view optimization. In *Computer Graphics Forum*, pages 29–43. Wiley Online Library, 2021. [1](#), [2](#), [3](#)
- [15] Xin Lai, Jianhui Liu, Li Jiang, Liwei Wang, Hengshuang Zhao, Shu Liu, Xiaojuan Qi, and Jiaya Jia. Stratified transformer for 3d point cloud segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8500–8509, 2022. [3](#)
- [16] Joo Chan Lee, Daniel Rho, Xiangyu Sun, Jong Hwan Ko, and Eunbyung Park. Compact 3d gaussian representation for radiance field. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21719–21728, 2024. [2](#)
- [17] Tianqi Liu, Guangcong Wang, Shoukang Hu, Liao Shen, Xinyi Ye, Yuhang Zang, Zhiguo Cao, Wei Li, and Ziwei Liu. Mvsgaussian: Fast generalizable gaussian splatting reconstruction from multi-view stereo. In *European Conference on Computer Vision*, pages 37–53. Springer, 2024. [2](#)
- [18] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. [1](#), [2](#)
- [19] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)*, 41(4):1–15, 2022. [2](#)
- [20] Julian Straub, Thomas Whelan, Lingni Ma, Yufan Chen, Erik Wijmans, Simon Green, Jakob J Engel, Raul Mur-Artal, Carl Ren, Shobhit Verma, et al. The replica dataset: A digital replica of indoor spaces. *arXiv preprint arXiv:1906.05797*, 2019. [5](#)
- [21] Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, et al. Nerfstudio: A modular framework for neural radiance field development. In *ACM SIGGRAPH 2023 conference proceedings*, pages 1–12, 2023. [6](#)
- [22] Anh-Thuan Tran, Hoanh-Su Le, Suk-Hwan Lee, and Ki-Ryong Kwon. Pointct: Point central transformer network for weakly-supervised point cloud semantic segmentation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 3556–3565, 2024. [3](#)
- [23] Qianqian Wang, Zhicheng Wang, Kyle Genova, Pratul P Srinivasan, Howard Zhou, Jonathan T Barron, Ricardo Martin-Brualla, Noah Snavely, and Thomas Funkhouser. Ibrnet: Learning multi-view image-based rendering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4690–4699, 2021. [2](#)
- [24] Xiaoyang Wu, Yixing Lao, Li Jiang, Xihui Liu, and Hengshuang Zhao. Point transformer v2: Grouped vector attention and partition-based pooling. *Advances in Neural Information Processing Systems*, 35:33330–33342, 2022. [3](#), [4](#)
- [25] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer v3: Simpler faster stronger. In *Pro-*

- ceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4840–4851, 2024. [3](#), [4](#)
- [26] Tianyi Xie, Zeshun Zong, Yuxing Qiu, Xuan Li, Yutao Feng, Yin Yang, and Chenfanfu Jiang. Physgaussian: Physics-integrated 3d gaussians for generative dynamics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4389–4398, 2024. [3](#)
  - [27] Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12–22, 2023. [5](#)
  - [28] Alex Yu, Vickie Ye, Matthew Tancik, and Angjoo Kanazawa. pixelnerf: Neural radiance fields from one or few images. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4578–4587, 2021. [2](#)
  - [29] Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip HS Torr, and Vladlen Koltun. Point transformer. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 16259–16268, 2021. [3](#), [4](#)
  - [30] Siting Zhu, Guangming Wang, Xin Kong, Dezhi Kong, and Hesheng Wang. 3d gaussian splatting in robotics: A survey. *arXiv preprint arXiv:2410.12262*, 2024. [1](#)
  - [31] Zihan Zhu, Songyou Peng, Viktor Larsson, Weiwei Xu, Hujun Bao, Zhaopeng Cui, Martin R Oswald, and Marc Pollefeys. Nice-slam: Neural implicit scalable encoding for slam. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12786–12796, 2022. [5](#)